
Advanced AI Interview Room System

ENTERPRISE TECHNICAL ARCHITECTURE & SYSTEMS DOCUMENTATION

Author: Simarpreet Singh

Date: June 22, 2026

Project Version: 1.2.5-enterprise

Stack: FastAPI, SQLite/PostgreSQL, Celery, Redis, PyTorch, SentenceTransformers, FAISS, Face-API.js, Groq Whisper

Document Classification: Technical Architecture & Reference Manual

1. Executive Summary & Project Goals

The **Advanced AI Interview System** addresses the scalability bottlenecks, high overhead, and lack of metrics inherent in conventional corporate interviewing. By packaging complex client-side computer vision models, neural semantic text indexers, and distributed audio processing microservices, the application acts as an automated mock interviewing partner and security auditor.

Rather than relying on human graders, the system processes responses through multiple layers. Real-time video emotion matrices and client gaze indicators log candidate compliance, while automatic text transcribers turn speech into structured dialogue paragraphs. The paragraphs are evaluated using Large Language Models (Gemini, GPT-4o, or local Llama 3) to score the candidate's technical competence, providing granular feedback. In the event of system or API disconnections, local fallbacks automatically handle grading and speech boundaries offline, ensuring seamless operation.

1.1 Document Roadmap

This architectural manual covers: (1) Executive Summary, (2) Deep Architecture & Data Flows, (3) Backend System File Decomposition, (4) Machine Learning & Intelligence Modules, (5) Security Policies & Anti-Cheat Protocols, (6) Database Relational Schemas, (7) REST & WebRTC API Catalog, (8) Production Deployment Steps, and (9) Verification & Testing Schemes.

2. Technical Stack & Architectural Layout

The system has been developed with a clean separation of concerns. The client browser handles UI layout, camera rendering, and initial face vector mapping via TensorFlow.js weights. The server layer runs FastAPI for routing, database interfaces, and background queue tasks.

2.1 Technologies Matrix

Component Group	Tech / Package	Architectural Responsibility
Routing Framework	FastAPI / Uvicorn	Asynchronous high-performance endpoint router for registration, logins, uploads, and status retrievals.
Speech Processing	Groq API / Whisper-large	Transforms uploaded audio clips into highly accurate, structured candidate transcripts.
Facial Analytics	Face-API.js	Client-side neural network tracking eyes, expressions, and frame focus points in real-time.
Evaluation Models	Gemini 1.5 Flash / GPT-4o	Evaluates answer accuracy, technical depth, and provides scoring and textual feedback.
Database Layer	PostgreSQL / SQLite	Stores applicant data, session reports, credentials, and integrity logs.
Background Work	Celery / Redis	Handles file transfers and speech transcription asynchronously.
Semantic Search	FAISS / SentenceTransformers	Indexes the question bank using vector embeddings for skill-matched retrieval.
Document Engine	ReportLab SDK	Generates custom PDF summaries for candidate records and admin downloads.

3. Core Backend Decomposition

The system's modular architecture separates core server logic from database schemas and utility files.

- **main.py:** Acts as the project orchestrator. Handles user sessions, JWT token generation, resume uploads, and WebRTC signaling. Also maintains real-time synchronization between the primary database (PostgreSQL) and the local fallback database (SQLite).
- **database.py:** Initializes the SQLAlchemy engine, configures connection pooling, and handles automatic adjustments for PostgreSQL connection strings (e.g. converting `postgres://` to `postgresql://`).
- **models.py:** Defines the database schema using SQLAlchemy ORM, including relationships between the `User` and `InterviewSession` tables.
- **storage.py:** Manages file uploads to Cloudinary for media storage, with automatic fallbacks to local directory storage (`/static/videos`) if cloud credentials are not found.
- **migrate_to_postgres.py:** A utility script that migrates local user and session data from SQLite to PostgreSQL, adjusting database sequences to prevent ID collisions on future inserts.
- **generate_pdf_report.py:** Compiles system data and generates PDF summaries for candidate reviews.

3.1 Core Database Schemas

Data persistence is maintained across two main relational tables: users for profile information and interview_sessions for recording candidate responses.

3.2 Table Schema: users

Field Name	Type	Description
id	Integer (PK)	Unique sequence id.
username	String	Unique identifier for logins.
password	String	Plaintext password.
status	String	Candidate state: Pending, Approved, Rejected.
access	String	Access privileges: grant, revoke.
resume_path	String	File path or Cloudinary URL of the resume PDF.
skills	String	Parsed candidate skills list.
email / phone	String	Contact details extracted from the resume.
integrity_notes	String	Timestamped list of focus violations during the session.
age	Integer	Candidate's age.

3.3 Table Schema: interview_sessions

Field Name	Type	Description
id	Integer (PK)	Unique sequence id.
username	String (FK)	References users.username.
date	String	Session timestamp.
question	String	Interview question prompt.
answer	String	Speech-to-text response transcript.
emotion	String	Dominant candidate emotion detected during the answer.
score	Float	Evaluation score (0.0 to 100.0).
video_url	String	Path/URL to the recorded video response.
evaluation_feedback	String	Granular text evaluation feedback from the LLM.

4. Machine Learning & NLP Architecture

A key design objective was optimizing AI inference times. The backend utilizes lightweight, container-safe models for local preprocessing before querying external LLM endpoints.

4.1 VAD (Voice Activity Detection) - modules/vad_detector.py

To reduce background noise interference, the system employs the pre-trained **Silero VAD** model using PyTorch hub integrations. The model analyzes normalized float32 audio segments at a target sample rate of 16kHz. If initialization fails, the script automatically falls back to a Root Mean Square (RMS) energy-based threshold checker:

```
rms = np.sqrt(np.mean(np.square(audio_chunk)))  
return rms > 0.025
```

4.2 Semantic Question Retrieval - modules/question_retriever.py

Instead of simple keyword matching, the question generator supports semantic queries using the `all-MiniLM-L6-v2` transformer and **FAISS** (Facebook AI Similarity Search). Skills extracted from resumes are embedded into vector vectors. A flat L2 index allows the system to query and retrieve semantically similar questions, ensuring candidates receive relevant prompts. On OOM-constrained hosts, setting `DISABLE_ML=1` or `RENDER=true` bypasses FAISS initialization to ensure stability on lower-tier servers.

4.3 Speech-to-Text Conversion - modules/whisper_transcriber.py

Candidate audio recordings are converted to text using the Groq Whisper API (`whisper-large-v3-turbo` model). The API processes raw audio chunks (converted to `audio/webm` format) to generate responses quickly, avoiding concurrency bottlenecks during live sessions.

4.4 Answer Evaluation Engine - modules/answer_analyzer.py

The evaluation engine grades candidate responses based on a hierarchy of available services:

- **Gemini 1.5 Flash:** The default evaluation model. Calls the Google Generative Language endpoints using a structured JSON schema, prompting the model to return both a grade and constructive feedback.
- **OpenAI GPT-4o:** The secondary model fallback, activated if Gemini API tokens are invalid or rate limits are reached.
- **Ollama Llama 3:** Enables local offline evaluations using local Llama 3 models if configured via the `USE\_OLLAMA=1` parameter.
- **Offline TF-IDF Cosine Similarity:** The final local fallback. Computes the cosine similarity between the question prompt and the candidate's response text using Scikit-Learn. It applies scoring modifiers based on word count bonuses and emotional status metrics (e.g. +10% for confidence, -5% for fear).

5. Security Protocols & Integrity Monitoring

To ensure the credibility of mock interview results, the platform implements multi-layered integrity monitoring:

- **Webcam Gaze Tracking:** Runs client-side Face-API.js models to monitor candidate eye alignment. If the candidate looks away or their face leaves the frame, the interface registers a violation.
- **Focus Switch Detection:** Uses document focus listeners to detect if the user switches browser tabs or windows, logging violations to the database.
- **JWT Authentication & Lockouts:** Authenticates candidate requests using JWT tokens valid for 24 hours. Admins can revoke access keys in the dashboard, immediately redirecting candidates to a lockout page (`terminated.html`).

6. Core REST API Endpoint Catalog

The backend endpoints route candidate submissions and support administrator dashboard configurations:

- **POST /api/register:** Registers new candidate accounts. Checks username availability and mirrors profile data to the SQLite backup database.
- **POST /api/login:** Validates user credentials and issues a secure JWT token for session authorization.
- **POST /api/upload_resume:** Accepts resume file uploads, calls the PDF parser to extract metadata, and returns a customized list of questions.
- **POST /api/analyze_answer:** Receives the webcam recording and candidate transcript. Evaluates the response via Gemini or the offline fallback, saving the score, emotion data, and video path to the database.
- **GET /api/get_result:** Allows candidates to retrieve their evaluation status, average score, emotion analysis, and AI-generated feedback.
- **GET /api/get_all_records:** Admin-only endpoint. Compiles all candidate profiles and interview history, merging database records on the fly.
- **POST /api/admin_action:** Allows administrators to approve or reject applications, grant/revoke system access, or delete records.
- **POST /api/log_violation:** Appends focus or tab violation notes to the candidate's integrity logs.
- **POST /api/webrtc/offer:** Establishes WebRTC signaling parameters to record remote webcam streams on backend servers.

7. Infrastructure, Deployment & Staging Setup

The application supports multiple deployment environments to fit both local development and scalable production setups.

7.1 Local Development (Staging / Offline Setup)

To run the system locally using the SQLite database and synchronous threading:

1. Create a Python 3.10 virtual environment: `python -m venv venv`
2. Activate the virtual environment: `.\venv\Scripts\activate` (Windows) or `source venv/bin/activate` (Unix)
3. Install dependencies: `pip install -r requirements.txt`
4. Configure environment variables in a `.env` file (DATABASE_URL, API keys, etc.)
5. Launch the application: `python launcher.py`

7.2 Production Environment (Docker & PostgreSQL Stack)

For production-ready deployments, configure `docker-compose.yml` to orchestrate a containerized stack:

- **db:** Runs a PostgreSQL 15 database instance to persist user and session data.
- **redis:** Runs a Redis broker instance to manage Celery task queues.
- **web:** Runs the main FastAPI application server, routing requests to the database and queue worker.
- **worker:** Runs the Celery worker queue to process audio conversions and uploads asynchronously.

Deploy the containerized stack with the build flag in detached mode: `docker-compose up --build -d`

8. Quality Assurance & Verification Plan

System reliability was verified across multiple stages:

- **■ Resume Extraction Testing:** Validated regex parsing constraints against sample resume PDFs to ensure metadata (names, emails, skill tags) are extracted accurately.
- **■■■ Model Fallback Validations:** Tested the backend evaluation flow with missing or invalid API keys. Verified the system automatically redirects requests to local TF-IDF grading mechanisms.
- **■ Browser & WebRTC Diagnostics:** Verified microphone and webcam capture interfaces across various browsers, ensuring WebRTC media chunks are compiled and uploaded correctly.
- **■ Database Sync Verification:** Confirmed database synchronization scripts migrate local SQLite records to production PostgreSQL instances without losing data.

End of Enterprise Architectural Report